Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 1

The OS as a Resource Manager

Track A: The Real OS, from the Outside (Linux)

Elaborated by:
st. gr. FAF-243
Morari Veaceslav

Verified by:
Dumitrascu Marius

Chişinău – 2026

Contents

1 Introduction **2**

2 System Information **2**

3 Files and Directories **2**
3.1 Observations 3
3.2 Interesting Command Output 4

4 Processes **4**
4.1 Observations 4
4.2 Interesting Command Output 5

5 Memory Management **5**
5.1 Observations 5
5.2 Interesting Command Output 6

6 Devices and Storage **6**
6.1 Observations 6
6.2 Interesting Command Output 7

7 Conclusion **7**

1. Introduction

The purpose of this laboratory work is to observe how an operating system manages the main resources of a computer system: files, processes, memory and devices.

The experiment was performed using Ubuntu running under Windows Subsystem for Linux 2 (WSL2) instead of the provided virtual machine. WSL2 runs a real Linux kernel, so all commands behave the same way as on a normal VM, although some values, such as disk names and the amount of memory, are different. During the laboratory work, the four major areas of resource management were investigated one by one, and for each of them the questions of the "Observe" box were answered.

2. System Information

The following commands were used to identify the current user, the operating system kernel and the system uptime:

```
whoami
uname -a
uptime
```

The system reported the following information:

```
veacheslavv

Linux Veacheslav 6.18.33.2-microsoft-standard-WSL2
#1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026
x86_64 GNU/Linux

16:41:40 up 3:24, 2 users,
load average: 0.04, 0.05, 0.07
```

The OS knows me as the user `veacheslavv`. The machine is managed by Linux kernel 6.18, a version built by Microsoft for WSL2, on a 64-bit x86 processor. The system had been running for 3 hours 24 minutes, and the load average close to 0 shows that the processor was almost idle.

3. Files and Directories

In the first part of the laboratory work, the Linux file system was explored and basic file-management operations were performed. The following commands were used:

```
pwd
ls -la /
ls -la ~
cd /etc && ls | head
```

A demonstration directory and several files were then created, copied, renamed and removed:

```
mkdir demo && cd demo
```

```
echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
```

Finally, the permissions of the file were changed:

```
ls -l note.txt
chmod 600 note.txt
ls -l note.txt
chmod 644 note.txt
```

3.1. Observations

1. The files created during the experiment were owned by user `veacheslavv` and belonged to group `veacheslavv`. The OS automatically makes the user who creates a file its owner.

2. After executing

   ```
   chmod 600 note.txt
   ```

   the file permissions became:

   ```
   -rw-------
   ```

   The first character, `-`, indicates that `note.txt` is a regular file (a directory would show `d`).

   The next three characters, `rw-`, indicate that the owner can read and write the file but cannot execute it.

   The following `---` indicates that members of the group have no permissions, while the final `---` indicates that other users also have no permissions. The number 600 is these rights in octal: read = 4, write = 2, execute = 1, so the owner gets $4+2=6$ and the group and others get 0. After `chmod 644` the permissions became `-rw-r--r--` again, so everyone could read the file.

3. Two directories directly under the root directory `/` are:

   - `/etc` – contains system-wide configuration files (for example `apt`, `adduser.conf`, `bash.bashrc`).
   - `/home` – contains users' home directories (mine is `/home/veacheslavv`).

3.2. Interesting Command Output

The command `ls -la ~` showed the following line for one of the files in my home directory:

```
-rw------- 1 veacheslavv veacheslavv 72 Sep 27 13:47 .git-credentials
```

This file stores saved git passwords, and it already has the permissions `-rw-------`, so only its owner can read it. This is exactly the protection the OS gives through file permissions. I also noticed that `/proc` and `/sys` have size 0 in `ls -la /`, because they are not real directories on the disk: the kernel creates their contents on the fly.

4. Processes

The second part focused on process management. The currently running processes were inspected using:

```
ps aux | head
ps aux | wc -l
top
```

A new process was created in the background:

```
sleep 300 &
```

It was inspected using:

```
jobs
ps -ef | grep sleep
```

The process was then terminated using the `kill` command. The Linux pseudo-filesystem `/proc` was also used to inspect the information maintained by the kernel about the process:

```
sleep 300 &
PID=$!
ls /proc/$PID/
cat /proc/$PID/status | head -20
kill $PID
```

4.1. Observations

1. Process number 1 has PID 1. On the tested system it was `systemd` (`ps` shows it as `/sbin/init`, which is a link to systemd). It is the first user-space process started by the kernel; it starts and manages all system services and adopts processes whose parent has died. Under WSL there is also a small process `/init` with PID 2, Microsoft's helper that connects Linux with Windows.

2. Approximately **38 processes** were running while the Linux environment was idle (`ps aux | wc -l` returned 39 lines, one of them the header). `top` reported 37 tasks: 1 running and 36 sleeping, with the CPU 98% idle.

3. For the test `sleep` process, the file `/proc/3411/status` contained:

```
State:  S (sleeping)
```

   This means that the process existed but was waiting for its timer instead of actively using CPU time. The scheduler does not give it the processor until it wakes up.

4.2. Interesting Command Output

The process status contained the following information:

```
Name:   sleep
State:  S (sleeping)
Pid:    3411
PPid:   2892
Uid:    1000    1000    1000    1000
VmPeak:    16116 kB
VmSize:    16116 kB
```

The parent process `PPid:  2892` is my `bash` shell, so every program started from the terminal becomes a child of the shell, and `Uid:  1000` is my user, which is why I was allowed to kill the process. I also saw two different endings of a process: the first `sleep` finished by itself after 300 seconds (the shell printed `Done`), while the second one was stopped by `kill`, which sends it the termination signal (`Terminated`).

5. Memory Management

The third part of the laboratory work investigated how Linux manages physical memory and swap space. The following commands were executed:

```
free -h
cat /proc/meminfo | head -6
```

The resident memory used by a simple `sleep` process was inspected with:

```
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status
kill $PID
```

5.1. Observations

1. The system had approximately **15 GiB of total RAM**, with about **12 GiB free** at the time of observation (13 GiB available). The laptop has 32 GB of RAM, but WSL2 receives only half of it by default.

2. Swap is storage space on the disk that the operating system can use as an extension of physical memory when RAM is under pressure; pages that are not used right now are moved there. The tested system had **4.0 GiB of swap** configured, and none of it was being used during the experiment.

3. The `sleep` process used approximately **7888 kB** of resident memory according to its `VmRSS` value. This surprised me, because `sleep` does nothing. The reason is that RSS also counts shared libraries such as libc, which are really shared with many other processes, and in new Ubuntu versions `sleep` is part of one bigger coreutils program. The memory that belongs only to this process is much smaller.

5.2. Interesting Command Output

```
        total   used   free  shared  buff/cache  available
Mem:     15Gi  1.9Gi   12Gi   188Mi       1.0Gi       13Gi
Swap:   4.0Gi     0B  4.0Gi
```

The output demonstrates how the operating system divides physical memory between used memory, free memory and cached data. The "available" value is larger than "free" because Linux uses spare RAM as a cache for files and gives it back as soon as a program needs it.

6. Devices and Storage

The final experimental part focused on storage devices, mounted filesystems and device interfaces. The following commands were used:

```
df -h
lsblk
du -sh ~/os-lab1
ls -l /dev | head
mount | head
```

The device containing the root filesystem was additionally checked using:

```
findmnt /
```

6.1. Observations

1. The root filesystem `/` was mounted on:

   ```
   /dev/sde
   ```

   It uses the `ext4` filesystem. Under WSL2 this disk is virtual: it is one big file on the Windows drive, but Linux sees it as a normal 1 TB disk.

2. One example of a device interface found inside `/dev` is `/dev/console`. It represents the system console and provides a file-like interface for terminal input and output. The letter `c` at the start of its permissions (`crw-------`) means it is a character device, which transfers data byte by byte, while disks such as `sde` are block devices.

3. The UNIX principle "everything is a file" means that many operating system resources, including hardware devices (`/dev/sde`), the terminal (`/dev/console`) and

even running processes (`/proc/3411/status`), are exposed through file-like interfaces that programs can access with the same read and write operations.

6.2. Interesting Command Output

The `lsblk` command showed the block devices visible to Linux:

```
NAME  MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda     8:0    0 356.9M  1 disk
sdb     8:16   0 159.4M  1 disk
sdc     8:32   0     4G  0 disk [SWAP]
sdd     8:48   0 141.5M  0 disk
sde     8:64   0     1T  0 disk /mnt/wslg/distro
                                /
```

The output shows that `sdc` was used for swap, while `sde` contained the root filesystem. In addition, `df -h` showed that the Windows drives C: and D: are also visible inside Linux, mounted as `/mnt/c` and `/mnt/d`.

7. Conclusion

The operating system manages four major categories of resources: **files, processes, memory, and devices/storage**.

File management was observed using commands such as `ls -l` and `chmod`; process management was inspected using `ps`, `top` and `kill`; memory usage was examined using `free` and the `VmRSS` value in `/proc`; and storage devices were inspected using `lsblk` and `findmnt`.

The laboratory work demonstrated how Linux provides structured and controlled access to system resources and acts as an intermediary between running programs and the underlying hardware: programs never touch the hardware directly, but receive every file, process, memory page and device from the operating system.